

Python Moderno para Desarrolladores Java

Sesión 1: Tipado Gradual, Estructuras Nativas y Procesamiento Funcional

</> | Módulo: Sistemas de Gestión Empresarial (2º DAM)

</> | Enfoque: Del ecosistema Java a la arquitectura backend de software de gestión / Python 3.10+ idiomático.

Gemini Notebook

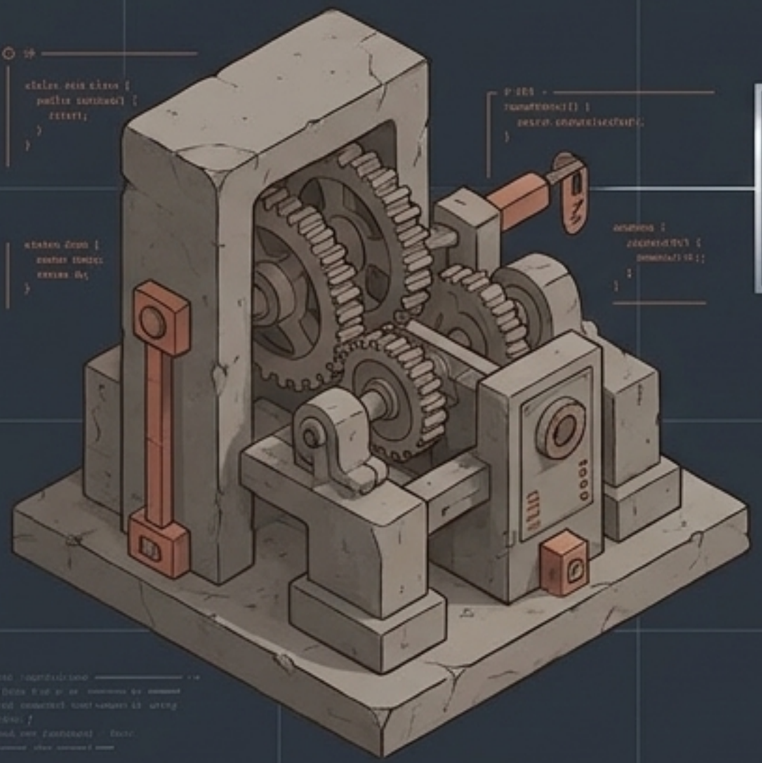
Hoja de Ruta de la Sesión (60 min)



Gemini Notebook

Java

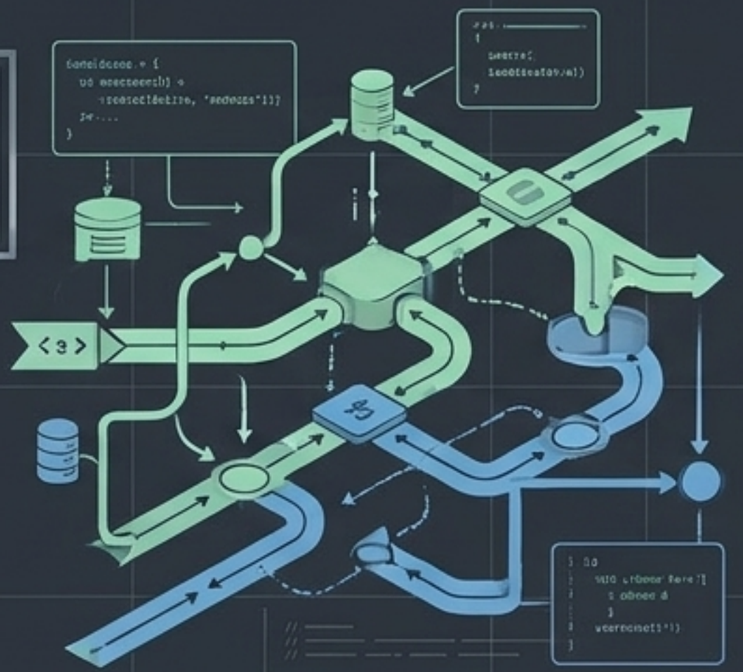
- Tipado nominal estricto (javac)
- Encapsulación obligatoria en clases
- Verbosidad ceremonial



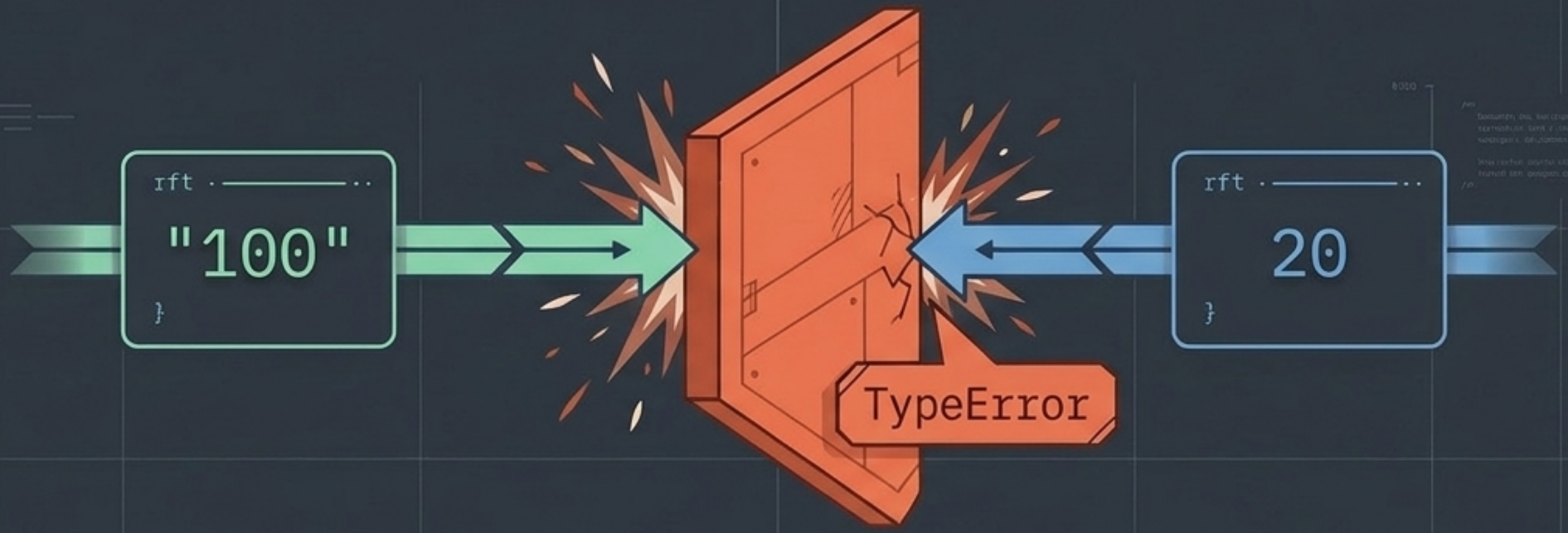
Simple es mejor que complejo

Python Moderno

- Tipado gradual (linters/mypy)
- Multiparadigma (funciones y módulos conviven con clases)
- Agilidad arquitectónica



Gemini Notebook



El Mito	La Realidad
Python no tiene tipos y propicia código frágil.	Python NO realiza coerción implícita. Sumar '100' + 20 detiene la ejecución inmediatamente.

El Estándar 3.10+: Tipado dinámico pero estrictamente fuerte, blindado por Type Hints (PEP 484/604).

Gemini Notebook

Transición de Tipos Primitivos

Java		Python 3.10+
int / Integer		**int** Precisión arbitraria sin overflow
double / Double		**float** Estándar IEEE 754 de 64 bits
boolean / Boolean		**bool** True / False con mayúscula obligatoria
String		**str** Inmutable Unicode UTF-8
null		**None** NoneType, objeto singleton del sistema

Gemini Notebook

Anotación de Funciones con Type Hints

Java

```java
public double aplicarDescuento(double base, double porcentaje)
```

Python 3.10+

```python
def aplicar_descuento(base: float, porcentaje: float) -> float
```

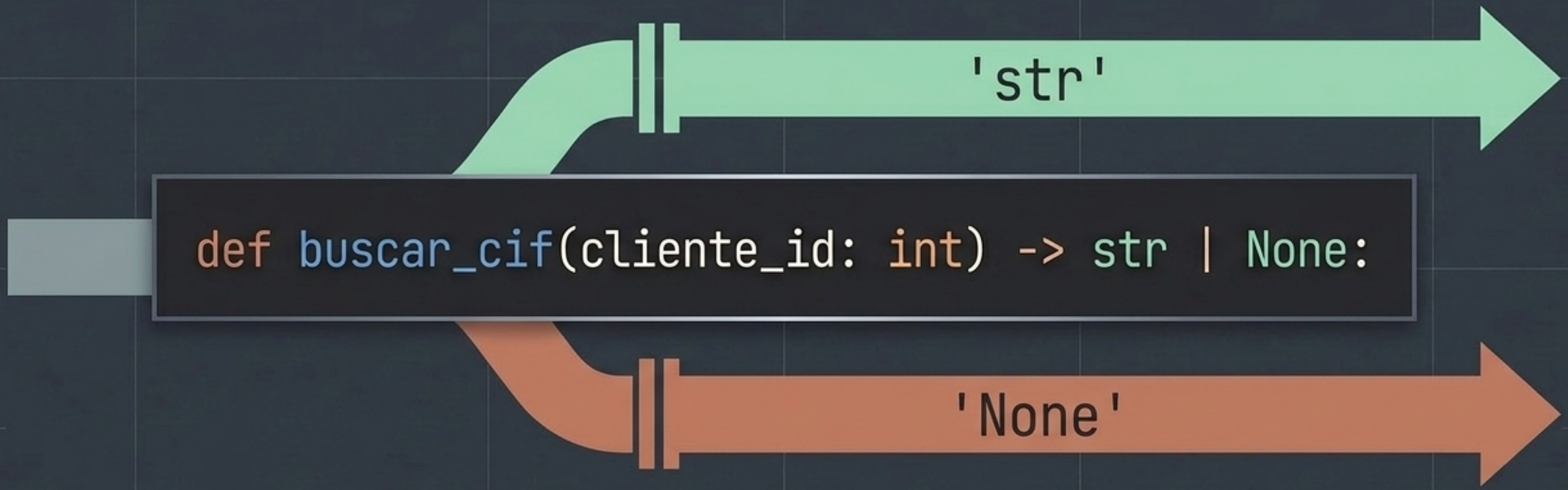
define el tipo en parámetros

define el tipo de retorno al final (lectura natural)

Nombres siempre en snake_case (Norma PEP 8)

Gemini Notebook

Nulabilidad Explícita: El Fin del NullPointer



- De Java 'Optional<String>' al Operador Tubería ('|')
- Contrato explícito de nulabilidad nativo en la firma. El IDE/linter obligará a validar con 'if' antes de usar la variable, sin necesidad del módulo 'typing'.

Gemini Notebook

Uniones y Colecciones Genéricas Nativas

Unión de Tipos



```python
def consultar_factura(identificador: int | str) -> dict
```

Una firma que absorbe lo que en Java requeriría dos métodos sobrecargados.

Genéricos Nativos (Python 3.9+)

~~`from typing import List, Dict`~~

Listas: `list[str]`
Diccionarios: `dict[str, float]`

Colecciones genéricas directas, sin imports artificiales.

Gemini Notebook

Tuplas: Inmutabilidad Ligera para Registros



Definición:

```
tuple[int, int, float] = (1092, 4, 18.50)
```

Garantía de integridad:

```
tupla[0] = 999
```

TypeError

Concepto: Inmutabilidad ligera para registros.
Alternativa ultra-rápida a instanciar clases completas.

Gemini Notebook

Diccionarios: DTOs Ágiles y Acceso Defensivo

```
articulo = {'sku': 'PORT-15', 'stock': 42, 'precio': 799.0}
```

Acceso Estricto




```
dict['sku']
```

Falla ruidosamente (KeyError) si el ERP pierde la clave crítica.

Acceso Defensivo



```
dict.get('descuento', 0.0)
```

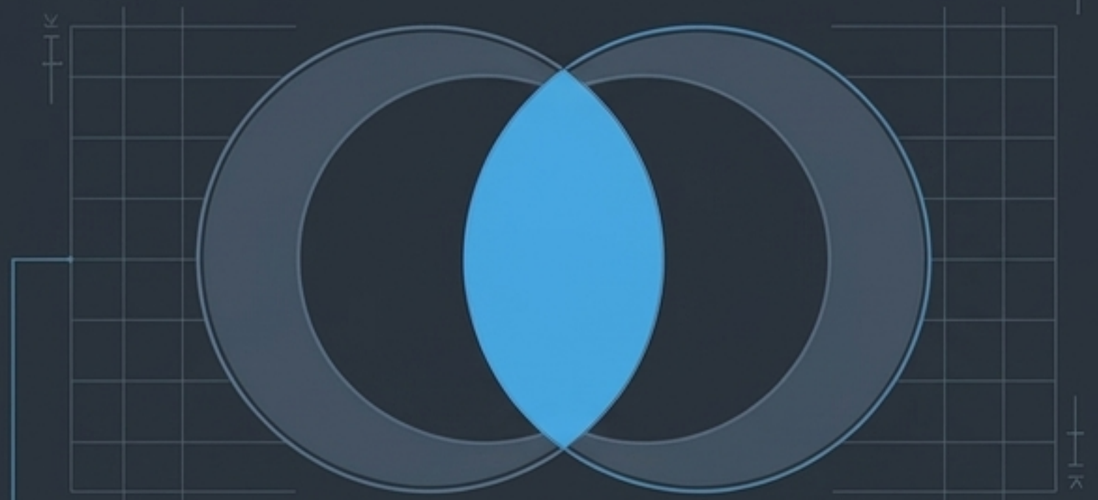
Proporciona un valor por defecto seguro (*fallback*) sin bloques try/except.

Gemini Notebook

Conjuntos (set): Operaciones de Almacén

Colecciones sin orden ni duplicados, optimizadas para búsquedas instantáneas O(1)

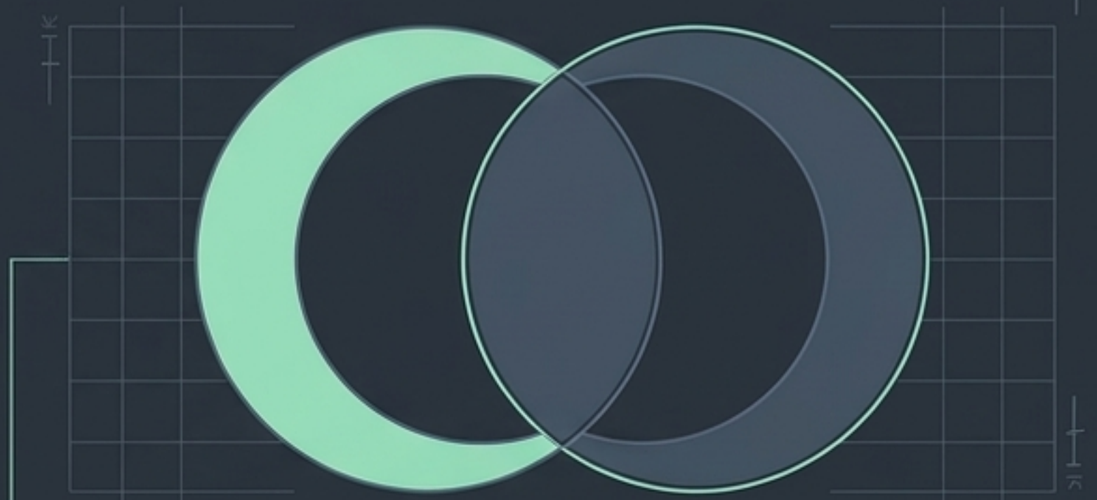
1 Intersección (&)



```
comunes = almacen_norte & almacen_sur
```

Productos que existen en ambos almacenes.

2 Diferencia (-)

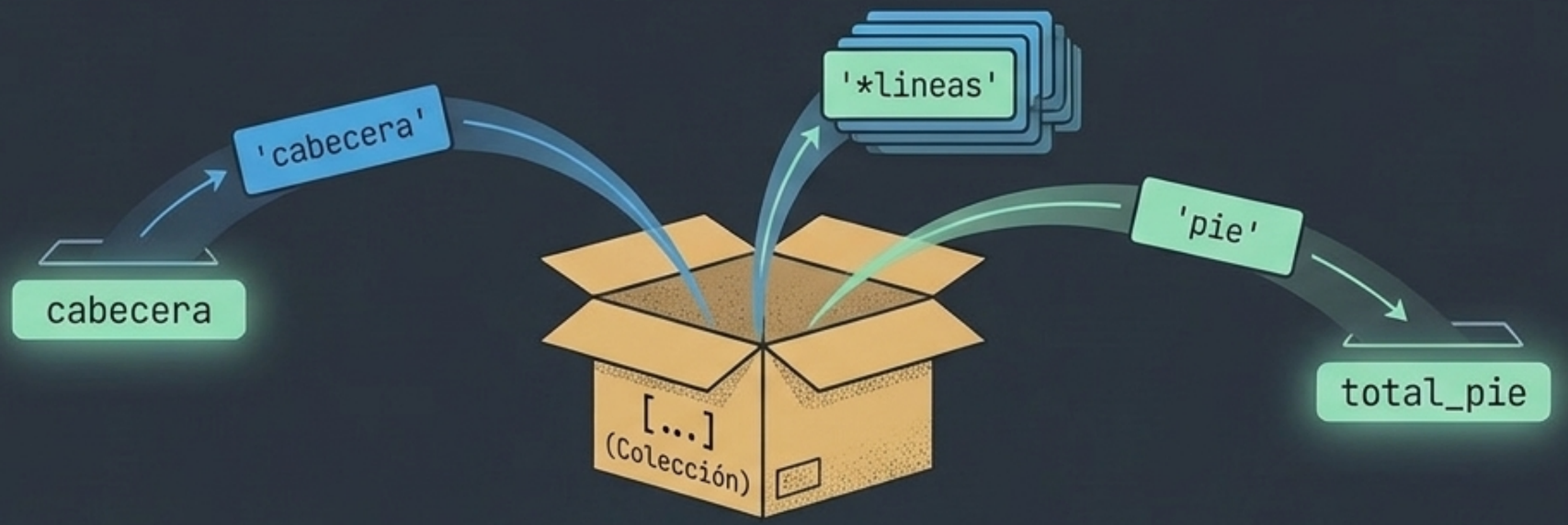


```
exclusivos = almacen_norte - almacen_sur
```

Comparar inventarios para ver qué falta en un solo paso matemático.

Gemini Notebook

Desempaquetado (Unpacking) y Asignación Múltiple



Asignación múltiple:

```
codigo_impuesto, tipo_recargo = ('IVA_GENERAL', 21.0)
```

Intercambio atómico (adiós variables temporales):

```
a, b = b, a
```

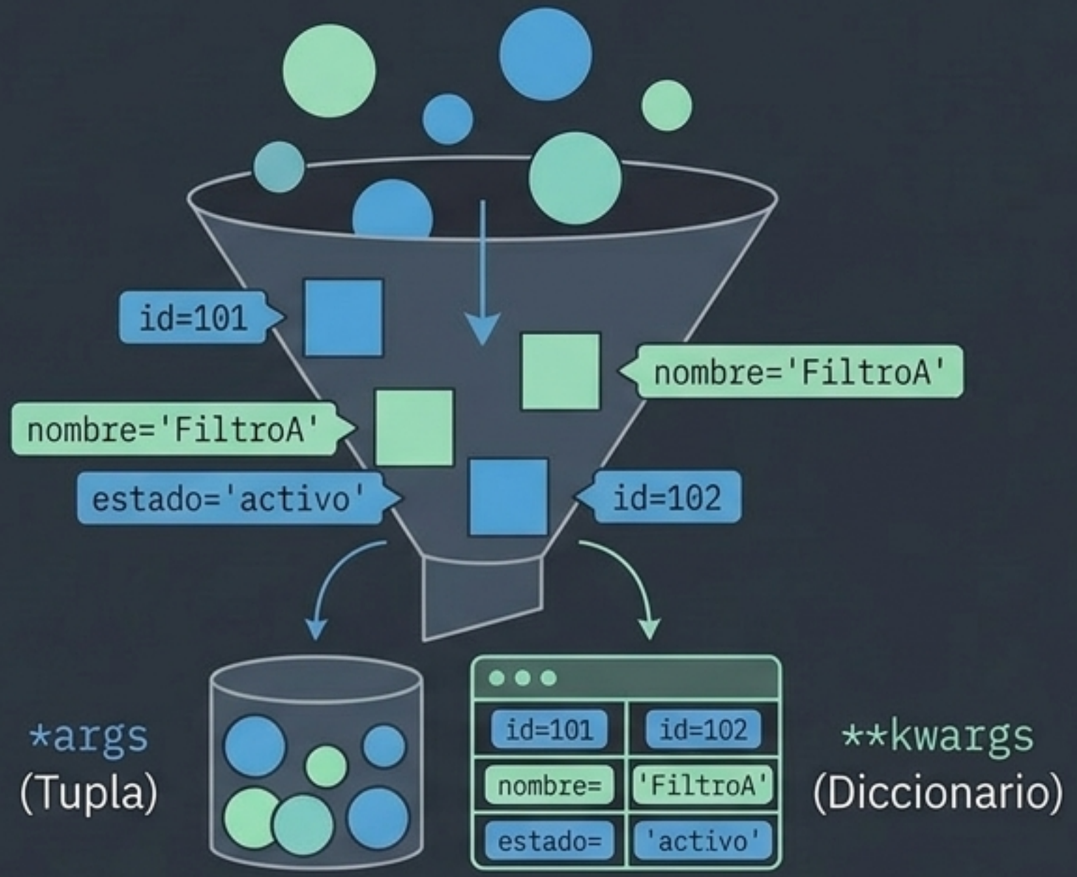
El poderoso comodín Resto (*):

```
cabecera, *lineas_detalle, total_pie = [...]
```

Eliminación total de variables temporales y métodos tediosos para extraer fragmentos de colecciones.

Gemini Notebook

Métodos Flexibles: *args y **kwargs



- '*args': Agrupa argumentos posicionales variables en una tupla.

- '**kwargs': Agrupa argumentos nombrados variables en un diccionario.

Ideal para inyectar filtros dinámicos o criterios de búsqueda en sistemas de gestión (ERPs) sin modificar las firmas de los repositorios.

Gemini Notebook

De Java Streams a List Comprehensions

Filtrar pedidos > 100 y aplicar 21% IVA

```
importes.stream()
    .filter(imp -> imp > 100.0)
    .map(imp -> imp * 1.21)
    .toList()
```

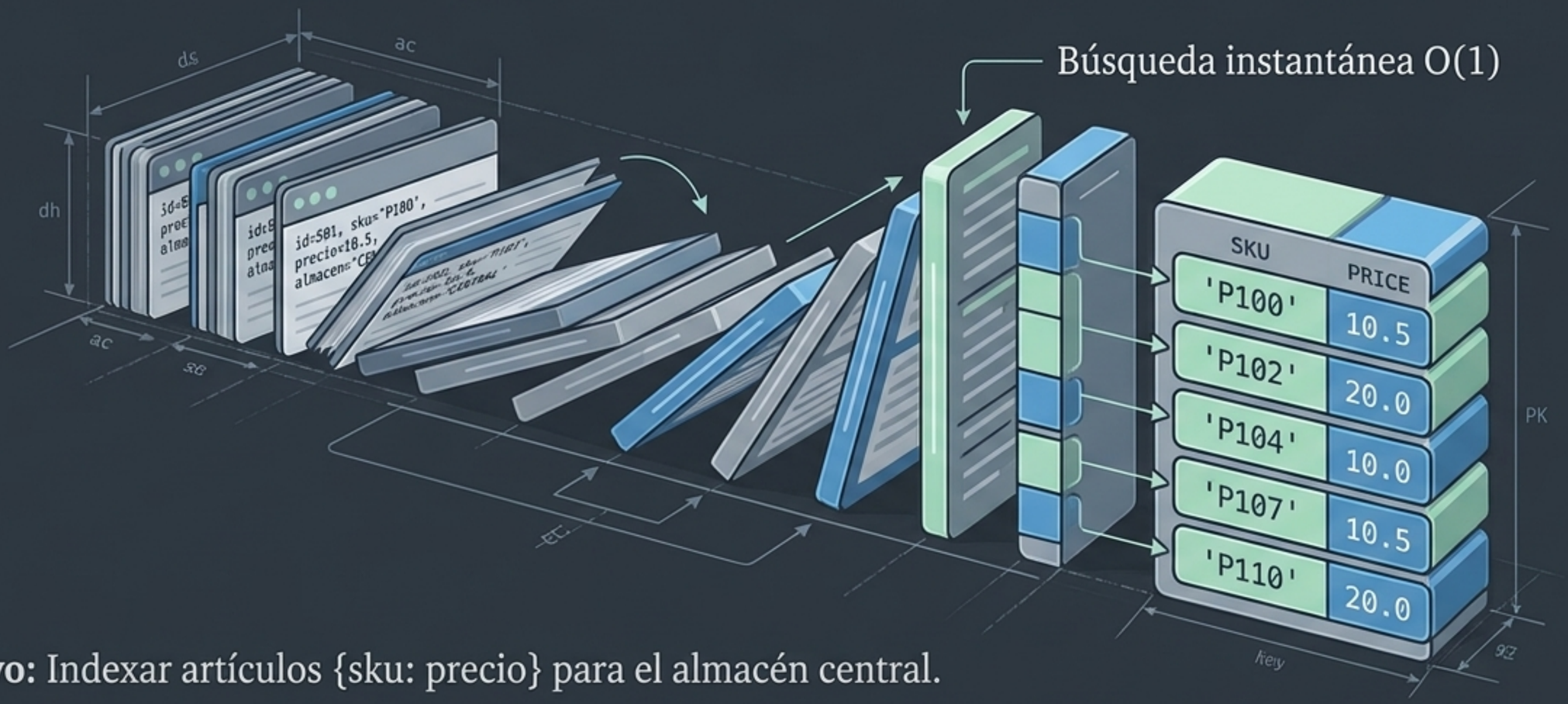
```
[imp * 1.21 for imp in importes if imp > 100.0]
```

```
[<expresión_resultado> for <elemento> in <colección> if <condición>]
```

Las comprensiones no son simples bucles, son expresiones funcionales optimizadas en C.

Gemini Notebook

Dict Comprehension: Indexación en Memoria



Objetivo: Indexar artículos {sku: precio} para el almacén central.

```
{item['sku']: item['precio'] for item in catalogo if item['almacen'] == 'CENTRAL'}
```

Reemplazo elegante y directo del complejo 'Collectors.toMap()' de Java mediante sintaxis nativa con llaves {}.

Gemini Notebook

Micro-reto Práctico: Procesador de Pedidos

Regla de Oro: ¡Prohibido usar bucles 'for' acumulativos tradicionales (res = [])!
Solo Type Hints y Comprensiones funcionales.



10-15 min

Input Data (ERP Feed)

```
ventas = [
    {"id": "PED-01", "total": 1250.0},
    {"id": "PED-02", "total": 80.0},
    {"id": "PED-03", "total": 3500.0},
    {"id": "PED-04", "total": 100.0},
    {"id": "PED-05", "total": 220.0}
]
```

Challenge (Fill-in-the-blank)

```
# 1. Filtrar pedidos relevantes por umbral (devuelve IDs)
def pedidos_relevantes(datos: list[dict], umbral: float) -> list[str]:

    ... # Implementación usando list comprehension
```

```
# 2. Indexar pedidos por ID para búsqueda rápida (ID -> Total)
def indexar_por_id(datos: list[dict]) -> dict[str, float]:

    ... # Implementación usando dict comprehension
```

Gemini Notebook

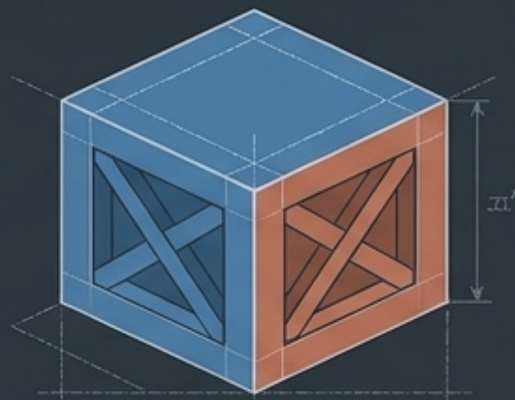
Solución del Reto y El Siguiente Nivel

```
return [str(v['id']) for v in datos if float(v['total']) > umbral]
return {str(v['id']): float(v['total']) for v in datos}
```



El Problema: Los diccionarios no garantizan campos ni validan tipos internamente.





@dataclass

Próximo paso (Sesión 2): Transición a la Programación Orientada a Objetos moderna y sin boilerplate con '@dataclass(frozen=True)'.

Gemini Notebook